This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

Online Multi-Workflow Scheduling under Uncertain Task Execution Time in IaaS Clouds

Jiagang Liu, Ju Ren, *Member, IEEE,* Wei Dai, Deyu Zhang, *Member, IEEE,* Pude Zhou, Yaoxue Zhang, *Senior Member, IEEE,* Geyong Min, *Member, IEEE,* Noushin Najjari

Abstract— Cloud has become an important platform for executing numerous deadline-constrained scientific applications generally represented by workflow models. It provides scientists a simple and cost-efficient method of running workflows on their rental Virtual Machines (VMs) anytime and anywhere. Since pay-as-you-go is a dominating pricing solution in clouds, extensive research efforts have been devoted to minimizing the monetary cost of executing workflows by designing tailored VM allocation mechanisms. However, most of them assume that the task execution time in clouds is static and can be estimated in advance, which is impractical in real scenarios due to performance fluctuation of VMs. In this paper, we propose an onliNe multi-workflOw Scheduling Framework, named NOSF, to schedule deadline-constrained workflows with random arrivals and uncertain task execution time. In NOSF, workflow scheduling process consists of three phases, including workflow preprocessing, VM allocation and feedback process. Built upon the new framework, a deadline-aware heuristic algorithm is then developed to elastically provision suitable VMs for workflow execution, with the objective of minimizing the rental cost and improving resource utilization. Simulation results demonstrate that the proposed algorithm significantly outperforms two state-of-the-art algorithms in terms of reducing VM rental costs and deadline violation probability, as well as improving the resource utilization efficiency.

Index Terms—Cloud Computing, Workflow Scheduling, VM Rental Cost, Uncertain Task Execution Time, Multiple Workflows.

1 Introduction

With the rapid development of big data analytics, data processing applications are becoming increasingly complex and generally comprised of hundreds or even thousands of compute-intensive and data-intensive tasks with dependent relationships [1]. A popular method to represent such applications is to adopt the direct acyclic graph (DAG) structure model, known as workflow, which has been widely applied in many scientific communities, such as astronomy, high-energy physics, bioinformatics and earthquake science. These scientific workflows are usually deadline-constrained, and require massive and distributed computing resources for timely executions [2], [3]. Traditional high performance computing platforms [4], such as clusters and grids [5], have very high maintenance cost and cannot efficiently adapt to the surge of computing resources required for execution of the workflows with random arrivals. This consequently motivates the development of new computing platforms that are capable of provisioning elastic computing resources for processing the randomly submitted workflows. Cloud computing provides such feasible and efficient solution for deployment of scientific workflows. Particularly, Infrastructure-as-a-Service (IaaS) clouds offer many types of pre-configured computing resources named virtual machines (VMs) for leasing, allowing customers to access nearly unlimited VMs on-demand based on the "pay-as-you-go" pricing scheme. Different types of VMs can be dynamically acquired and released for processing various workflows, and generally charged per time frame or billing period. For example, Amazon's Elastic Compute Cloud (EC2)[1] charges customers by an hourly-based pricing model. Consequently, optimizing the system performance, as well as minimizing the VM rental cost, by elastically provisioning VM resources for executing workflows has become a research trend in cloud computing.

As a result, there have been numerous research works focusing on optimally allocating suitable computing resources for the continuously submitted workflows. Since the workflow scheduling problem is a well-known NP-hard problem, it has been widely studied in clusters and grids [6]–[8] with the objectives of minimizing total execution time or maximizing energy efficiency. However, when scientific workflows are deployed in IaaS clouds, the research focus changes to how to minimize the total cost of VM leasing and guarantee the deadline constraints of each workflow. To achieve this goal, a tailored workflow scheduling algorithm should carefully consider two primary characteristics of IaaS clouds, i.e., elastic VM resources and pricing scheme based on billing periods. For the former, the scheduling algorithm should first select a suitable number and type of VMs from the resource pool to execute workflows and meet their deadline constraints. While for the latter, the algorithm should minimize the total cost of VM leasing under a time-cost trade-off by making full use of the time units charged on the VMs to finish the workflows. With these considerations, extensive workflow scheduling algorithms have been proposed recently (e.g., [9]–[15]), which are

- *Jiagang Liu, Ju Ren, Deyu Zhang, Pude Zhou and Yaoxue Zhang are with the School of Information Science and Engineering, Central South University, Changsha, China, 410083. Jiagang Liu is also with the School of Computer and Information Science, Hunan Institute of Technology, Hengyang, China, 421002. E-mails: {jiagliu, renju, zdy876, pude.zhou, zyx}@csu.edu.cn.*
- *Wei Dai is with University of Arkansas at Little Rock, USA. E-mail: wxdai@ualr.edu.*
- *Geyong Min, Noushin Najjari are with the Department of Mathematics and Computer Science, University of Exeter, Exeter, United Kingdom, EX4 4QF. E-mails:{g.min@exeter.ac.uk, n.najjari@gmail.com.}*
- *Ju Ren is the corresponding author of this paper.*

*Manuscript received *, *; revised August *, *.*

1. https://aws.amazon.com/cn/ec2/pricing/on-demand/

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

capable of optimizing the rental cost and utilization efficiency of VMs. However, vast majority of the existing algorithms focus on elastically acquiring VMs provisioned by the IaaS provider and designing scheduling plans for a single workflow (e.g., [11], [14]). It is inevitable that a huge number of idle time slots caused by the constraints of data dependence relationships should appear on leased VMs, irrespective of strategies designed for scheduling a single workflow. Due to the random arrival of scientific workflows, workflow scheduling algorithm should be able to dynamically allocate VM resources for workflows submitted stochastically. Thus, we can orchestrate tasks among workflows submitted during the same periods to properly share billing periods of VMs, such that both the wastage of idle time slots can be reduced and the efficiency of resource utilization can be improved.

In addition, most of the existing workflow scheduling algorithms proposed to provision elastic VM resources are based on the assumption that the task execution time of each workflow on a specific type of VM is deterministic and can be accurately estimated beforehand [11], [14], [15]. However, since the VMs established on infrastructure of real commercial IaaS clouds are incapable of maintaining persistent performance [16], [17], the actual task execution time may fluctuate frequently, leading to considerable task execution delays. The delays also affect successor tasks under the data dependencies. Moreover, the effect will continuously accumulate until the workflows are complete, which may lead to the deadline violation for the workflows [18]. Therefore, besides the target of minimizing the VM rental cost, new challenges arise in controlling the propagated impact of the fluctuated task execution time of each workflow in IaaS clouds and reducing the deadline violation probability.

In this paper, we propose an onliNe multi-workflOw Scheduling Framework, named NOSF, for IaaS clouds. By dynamically allocating VM resources for randomly arrived workflows based on the online status of scheduled tasks, NOSF not only captures the stochasticity of task execution time on VMs, but also mitigates the propagated impacts of the fluctuated task execution time on each VM. The scheduling process of workflows in NOSF consists of three phases: workflow preprocessing, VM allocation and feedback process. The first phase determines the priorities of tasks in each arrived workflow in the light of its arrival time, and then transforms the deadline of the workflow to multiple subdeadlines for its tasks. The second phase heuristically allocates a VM with idle billing time for the task ready for scheduling to achieve minimum cost growth and to improve utilization efficiency. Moreover for the purpose of prohibiting the propagated impacts of the fluctuated task execution time on the unscheduled successor tasks, this phase only allows one task mapped to a VM to wait on the VM every time. The last phase is to adaptively adjust the originally assigned priorities and subdeadlines of the tasks ready for scheduling, such that the deadline of the workflow can be well met. Based on NOSF, we propose a deadline-aware heuristic algorithm to minimize the total VM rental cost and meet the deadline of each workflow. Specifically, the major contributions of this paper can be summarized as follows.

- We propose NOSF, an online multi-workflow scheduling framework, for IaaS clouds to dynamically allocate VM resources for randomly arrived workflows with full consideration of the stochastic task execution time caused by fluctuated VM performance. In NOSF, new arrival workflows can be processed in time of the execution of existing workflows, and the VMs with idle billing time can be shared between multi-workflows as much as possible for improving the utilization efficiency.
- We propose a deadline-aware heuristic algorithm, based on NOSF, to minimize the VM rental cost during the whole scheduling process.The proposed algorithm can reduce the deadline violation probability of the scheduled workflows, by adaptively adjusting the scheduling priorities and subdeadlines of the ready tasks online. Hence, the propagated impacts of the fluctuated task execution time can be mitigated.
- We conduct extensive simulations on realistic workflow datasets, and validate the superiority of NOSF, by comparing it to two existing algorithms, in terms of VM rental cost and deadline violation probability reduction (average 50.5% and 55.7%, respectively), as well as improving resource utilization efficiency (average 32.6%). Also, we evaluate the impact of the estimation method with a variation factor on the performance of the scheduling algorithm, and analyze how the algorithm can achieve the near-optimal VM rental cost and a better utilization efficiency based on the estimation method.

The remainder of the paper is organized as follows. Section 2 briefly reviews the related work, while Section 3 introduces the system model and problem formulation. Then, we present the details of the proposed NOSF in Section 4, and describe the deadline-aware heuristic algorithm and analyze its complexity in Section 5. Finally, performance evaluation is presented in Section 6, followed by a brief conclusion in Section 7.

2 Related Work

The workflow scheduling problem is a famous NP-hard problem. As a result, heuristic strategies are usually used to solve the challenge. In this section, we briefly review the existing heuristic strategies in IaaS clouds, and roughly divide them into three categories, including the scheduling under the deterministic execution time, the scheduling under the stochastic execution time and the scheduling for multiple workflows.

2.1 Scheduling under Deterministic Execution Time

Most existing works assume that the task execution time in VM is deterministic. For example, Abrishami et al. [9] propose the Partial Critical Paths (PCP) theory and design two deadline-constrained scheduling algorithms including IC-PCP and IC-PCPD2 in clouds, which can effectively reduce VM rental costs. Li et al. [14] extend the PCP algorithm to design a MCPCP algorithm for scheduling deadline-constrained workflows in multi-cloud, aiming to reduce VM rental costs. Wu et al. [19] develop a PCP-B^2 algorithm, which is an extension of the PCP considering the constraint of budget, to optimize the makespan.

The method of deadline assignment is also a critical strategy for workflow scheduling under the deterministic execution time. For instance, Cai et al. [11] focus on the nature of dependent task-batches in workflows, and present a deadline assignment method applied to task-batches. Furthermore, Calheiros et al. [20] separate workflow tasks into logical levels and proportionally divide the deadline over individual levels. However, the aforementioned scheduling strategies using the deadline assignment do not solve the challenge of VM performance fluctuations.

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

2.2 Scheduling under Stochastic Execution Time

Some existing workflow scheduling approaches consider the stochastic execution time on VMs with fluctuated performance. For example, Calheiros et al. [12] present an algorithm to replicate tasks, in order to mitigate the effects of VM performance fluctuations while matching workflow deadlines. Rodriguez et al. [18] propose a meta-heuristic approach to minimize the total VM rental cost for executing deadline-constrained workflows. After that, Rodriguez et al. [21] also propose a budget-driven algorithm to optimize the execution time of single workflow under the finer-grained pricing models. These algorithms estimate the task execution time based on the maximum computing capability of VMs, where task execution time is assumed to stochastically degrade within a range known in advance. However, our algorithms model the task execution time on VMs as a stochastic variable with the normal distribution.

In addition, the work [10] uses the same estimation method of the task execution time as used in this paper. Their adjustment scheme will update the subdeadline of a task-batch containing successors, once a VM completes a task. Sahni et al. [22] use a monitor loop to obtain the gap between the actual task start time and the estimated task start time, in order to make scheduling decisions in the next steps. But the time bound for scheduling each successor in Sahni's method is always the latest completion time determined initially. Besides, the works in [10], [22] utilize different methods that mitigate the effects of performance fluctuations. However, the aforementioned papers ignore the fact of sharing the billing time of leased VMs with multiple workflows and random arrival time.

2.3 Scheduling for Multiple Workflows

There are some existing scheduling strategies designed for multiple workflows. Rimal et al. [15] design a workflow scheduling policy, named CWSA, for the batch workload in multi-tenant clouds. Its objective is to minimize the completion time of the overall workflows, while neglecting the fluctuated performance of VMs in the policy. Moreover, in [23], authors propose a scheduling strategy to degrade the turnaround time of multiple workflows under the deterministic task execution time, for improving scheduling performance in cluster environments. Additionally, there are some works considering both multiple workflows and task execution time uncertainty. Chen et al. [24], [25] propose a heuristic strategy, which consists of proactive and reactive approaches, to schedule real-time multiple workflows with uncertain task execution time. In their strategies, nevertheless, the priority and decision parameters of each successor task which will be scheduled are throughout the original setting values during the scheduling.

In our work, NOSF divides the scheduling process of workflows into three phases, and dynamically allocates VM resources for randomly arrived workflows to share the billing time of leased VMs as much as possible. The proposed deadline-aware heuristic algorithm can adaptively adjust the scheduling priorities and subdeadlines of successor tasks ready for scheduling online to conquer the impacts of VM performance fluctuations.

3 Modeling and Problem Formulation

In this section, we describe cloud resources and workflow models. We also formulate the problem of workflow scheduling with the objective of minimizing the VM rental cost. To help understand, the common symbols used throughout this paper are listed in Table 1.

TABLE 1: Symbols and Meanings

Symbol	Definition
ξ_s	A workflow;
a^s	The arrival time of workflow ξ_s;
l^s	The deadline of workflow ξ_s;
t_i^s	The i-th task in workflow ξ_s;
$e_{p,q}^s$	A precedence constraint which indicates that t_p^s should be completed before t_q^s is started;
$d(e_{p,q}^s)$	The data transfer time associated with edge $e_{p,q}^s$;
$pred(t_i^s)$	All immediate predecessors of task t_i^s;
$succ(t_i^s)$	All immediate successors of task t_i^s;
I_k	The k-th active VM;
$B(I_k)$	The processing time weight of VM I_k;
$M^{est}(t_i^s)$	The earliest start time of task t_i^s;
$M^{eft}(t_i^s)$	The earliest completion time of task t_i^s;
λ_i^s	The stochastic execution time of task t_i^s on the VM with the highest ranking of computing services;
$w(\lambda_i^s)$	The approximate weight of the stochastic execution time λ_i^s;
$M^{lct}(t_i^s)$	The latest completion time of task t_i^s;
$\zeta(t_i^s)$	The completion time of task t_i^s;
$\varpi(t_i^s)_k$	The predicted start time of task t_i^s on VM I_k;
$\psi(t_i^s)_k$	The predicted execution time of task t_i^s on VM I_k;
$\varphi(t_i^s)_k$	The predicted completion time of task t_i^s on VM I_k;
Φ	The current system time;
$l_{sub}(t_i^s)$	The subdeadline of task t_i^s;
α	A variance factor of the task execution time;
β	A variance factor of the workflow deadline;
η	A variance factor to estimate the task execution time;

3.1 Cloud Resources and Workflow Models

For the cloud computing, cloud resources are comprised of virtualized services provisioned by IaaS cloud providers, which involve virtualized computing services and storage services [26]. The virtualized computing services can be categorized into various rankings based on computing capabilities supported by these services with diverse hardware configurations, such as CPUs, memory, storage, and network. Such that, we can use the ranking to describe the different capabilities of various VMs. Any partial-hour consumption of the leased VM is rounded up to the whole hour. Moreover, we only focus on the rental cost generated by the workflow scheduling strategy in this work, without considering the negotiated Service Level Agreements (SLAs) provided by IaaS cloud providers. With improvement of the virtualization technology, IaaS cloud platforms may give an illusion of unlimited provisional capabilities of VMs. Hence, our model supports unlimited VM instances and any rankings of computing services for customers. The set of VMs leased by a customer in an IaaS cloud are denoted as $\mathbb{I} = \{I_k | k \in [1, 2..]\}$, where k indicates the k-th leased VM. VMs with the same ranking can support the same computing capability and have the same rental price. Thus, VM I_k has a price factor associated with the service ranking, denoted as $price(I_k)$, which represents the cost of VM I_k in each billing

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

time unit. Additionally, each VM requires a boot time for its proper initialization and preparation for undertaking computation, while the VM is leased. Notice that a VM is only active when its billing time units are leased by a customer. In order to reduce energy consumption the VM shuts down immediately after it is released and is no longer in lease.

Except for computing services, the storage services offered by IaaS cloud providers can store the input/output data files of applications on the volume by leveraging a local storage device, e.g., Amazon Elastic Block Store (EBS)[2]. Commercial IaaS providers charge for storage services based on the amount of volume allocated, but the cost of storage is irrelevant to scheduling strategies. Thus, the file storage expense can be neglected by our model. Moreover, we consider all computing and storage services to be provisioned by the same data center. In this case, the average network bandwidth between different VMs is roughly equal, so that the time of data transfer only relies on the size of the data transferred [27]. Since commercial IaaS providers ignore the charge of data transfer in a single data center, communication expenses on different VMs are ignored in our model.

Task execution time on VMs depends on the task workload and computing capabilities of VMs [10], [15]. The execution time of a task on VMs with different rankings is not the same. Leasing a VM with a higher ranking means that a shorter execution time can be obtained but the VM rental cost will be increased. Therefore, we use a processing time weight associated with the ranking to describe the above features. Let the weight $B(I_k)$ of VM I_k be one if the execution time of a task can be minimized on this VM. For a specific VM I_k, $B(I_k)$ is defined as the ratio of the task execution time on I_k to the shortest execution time of the task.

In real cloud environments, VMs are incapable of maintaining persistent computing capabilities. Hence, the execution time of a task on a VM can fluctuate along with the variation of the computing capability of the VM. In the case, modeling of performance fluctuations of VMs can be transformed into a problem of modeling the task execution time with stochastic characteristics. The assumption that the task execution time is subject to normal distributions has been employed by many researchers [10], [28], [29]. In our work, the task execution time on a VM is also regarded as stochastic variables with independent and normal distributions.The expected value and variance of task execution time can be known by using some typical performance estimation techniques, such as historic tables [30], statistical prediction [31] and analytical modeling [32], in advance.

We use an approximate weight [33] as the estimation of the task execution time with the stochastic characteristic. Thus, given X as the task execution time, the approximate weight $w(X)$ is defined as

$$w(X) = \begin{cases} E(X) + \sqrt{D(X))}, & \frac{D(X))}{E(X)^2} \leqslant 1, \\ E(X) \cdot (1 + \frac{1}{\sqrt{D(X)}}), & \text{otherwise}, \end{cases} \quad (1)$$

where $E(X)$ and $D(X)$ represent the expected value and the variance of variable X, respectively. Let $N(\mu, \sigma^2)$ denote the normal distribution with expected value μ and variance σ^2. Since each value of X, $X \sim N(\mu, \sigma^2)$, satisfies $\sigma/\mu^2 \leqslant 1$, the weight of X is expressed as $w(X) = E(X) + \sqrt{D(X)}$.

A customer may randomly submit multiple workflows to an IaaS cloud provider. These submitted workflows may require

2. http://aws.amazon.com/ebs/

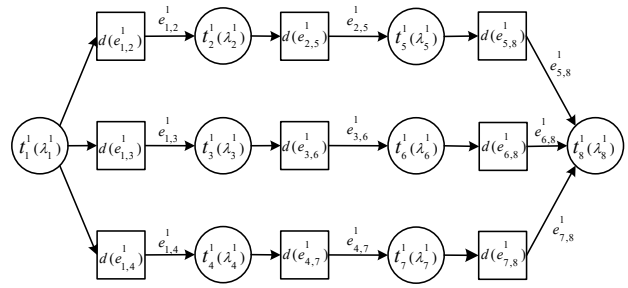


Fig. 1: An example of workflow modelled by DAG.

different software components for execution on the same operating system. We denote these workflows as a set of $\Omega = \{\xi_1, \xi_2, \cdots, \xi_w\}$, where $\xi_s = \{a^s, l^s, G^s\}$ denotes a workflow. a^s and l^s indicate the arrival time and the deadline of workflow ξ_s, respectively. $G^s(T^s, E^s, \Lambda^s, D^s)$ is a DAG graph that models workflow ξ_s. $T^s = \{t_1^s, t_2^s \cdots, t_n^s\}$ and $E^s = T^s \times T^s$ denote the set of tasks and edges, respectively. Then, $\lambda_i^s \in \Lambda^s$ denotes the stochastic execution time of task t_i^s with the normal distribution, while processing task t_i^s on the VM with the highest ranking of computing services. The directed edge $e_{i,j}^s \in E^s$ indicates the data dependency between task t_i^s and t_j^s. Thus, task t_i^s is the immediate predecessor of task t_j^s, while task t_j^s is the immediate successor of task t_i^s. Let $pred(t_i^s)$ and $succ(t_i^s)$ denote the set of the immediate predecessors and successors of task t_i^s, respectively. A task without predecessors is an entry task, denoted by t_{entry}^s, and a task without successors is an exit task, denoted by t_{exit}^s. Additionally, each edge $e_{i,j}^s$ has an associated time of data transfer $d(e_{i,j}^s) \in D^s$, which indicates the length of time required to transfer the output data from the VM executing t_i^s to the VM executing t_j^s. Notably, if both t_i^s and t_j^s are scheduled on the same VM, $d(e_{i,j}^s) = 0$. For each task t_j^s to be scheduled on VM I_k, I_k needs to be reserved before the output data from the immediate predecessors of task t_j^s starts to transfer. Then, VM I_k needs to keep active till all output data of VM I_k is transferred to the VMs on which the immediate successors of task t_j^s are to be scheduled. It should be noted that storage services provisioned by the IaaS cloud provider are independent of the lifetime of VMs, e.g. EBS. Thus, the VM, on which a task is to be scheduled, is not required to be active, as the input data is being transmitted to the associated volume. In addition, in our proposed model each VM can merely execute one task at a time and the execution of tasks are non-preemptive.

An example of the DAG (Fig. 1) consists of eight tasks from t_1^1 to t_8^1. t_1^1 and t_8^1 are the entry task and the exit task, respectively. The tasks are shown in the circles, while the transfer time associated with the output data are shown in rectangular shapes. For task t_2^1, its immediate predecessors and successors are $pred(t_2^1) = t_1^1$, $succ(t_2^1) = t_5^1$, respectively.

3.2 Problem Formulation

Our workflow scheduling aims to minimize the total VM rental costs for executing workflows, while satisfying the deadlines. Hence, in the following we define some notations.

While a VM completes a task, the completion time of the task and the start time of the task can be obtained, which are denoted as $\zeta(t_i^s)$ and $\tau(t_i^s)$, respectively. Furthermore, the makespan for executing a workflow determines whether the scheduling of the

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

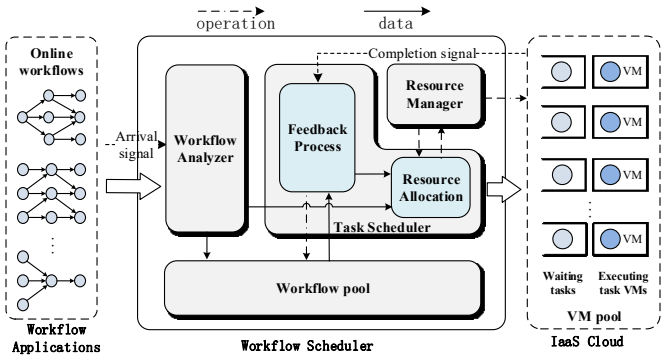


Fig. 2: The online workflow scheduling framework.

workflow can satisfy the deadline. We defined the makespan of workflow ξ_s as

$$MakeSpan_s = \max_{t_i^s \in T^s} \{\zeta(t_i^s)\} - a^s. \tag{2}$$

We use $\Gamma_{i,k}^s$ to denote the amount of incremental billing time units required to execute task t_i^s scheduled on VM I_k. Incremental billing time units are required, when VM I_k is newly leased to execute task t_i^s. Moreover, if executing task t_i^s does not require incremental billing time units, then $\Gamma_{i,k}^s = 0$. Hence, the optimization objective is defined as follows

$$\underset{x_{i,k}^s}{\text{Min}} \quad \sum_{k=1}^{|\mathbb{I}|} \sum_{s=1}^{w} \sum_{i=1}^{n} price(I_k) \cdot \Gamma_{i,k}^s \cdot x_{i,k}^s \tag{3}$$

$$\text{subject to:} \quad MakeSpan_s + a^s \leqslant l^s, \tag{4}$$

$$\zeta(t_p^s) + d(e_{p,i}^s) \leq \tau(t_i^s), \tag{5}$$

$$\sum_{k=1}^{|\mathbb{I}|} x_{i,k}^s \leqslant 1. \tag{6}$$

where $x_{i,k}^s$ is a binary variable indicating that t_i^s is scheduled on VM I_k. The binary variable is defined as

$$x_{i,k}^s = \begin{cases} 1, & \text{if } t_i^s \text{ is scheduled on } I_k, \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Constraint (4) indicates that the makespan of workflows should satisfy the deadlines. Constraint (5) indicates the data dependencies of tasks among workflows. The start time of task t_i^s should be greater than or equal to the sum of the completion time of immediate predecessor task t_p^s and the data transfer time $d(e_{p,i}^s)$. Also, constraint (6) ensures that each task can be scheduled once only.

4 NOSF: ONLINE MULTI-WORKFLOW SCHEDULING FRAMEWORK

In this section, we present each component of NOSF in detail. We also introduce the operations of every phase for workflow scheduling in NOSF.

4.1 Overview

The online multi-workflow scheduling framework (i.e., NOSF), shown in Fig. 2, acts as an interface between the IaaS cloud and customers. The core of NOSF is the workflow scheduler, which can use on-demand resource provisioning to select the required VMs for the execution of workflow tasks. The customer can submit new workflows with different deadline requirements to the workflow scheduler in time of the execution of existing workflows. Specifically, the workflow scheduler consists of a workflow analyzer, a resource manager, a task scheduler and a workflow pool. The workflow analyzer takes charge of analyzing the dependence relationships between workflow tasks and preprocessing workflows. The workflow pool accommodates all of unscheduled workflow tasks. The resource manager will negotiate with the IaaS cloud provider to adjust dynamically computing resource provisioning by acquiring and releasing VMs. Additionally, the task scheduler is responsible for identification of mappings between the workflow tasks and the offered VMs, in coordination with the resource manager. The task scheduler also takes charge of scheduling the workflow tasks on leased VMs according to the identified mappings.

The scheduling process of workflows in NOSF is divided into three phases, including workflow preprocessing in the workflow analyzer, resource allocation and feedback process. The scheduling process is motivated on receipt of two monitor signals, called the arrival signal and the completion signal, from the arrival of a submitted workflow and the completion of any executing tasks, respectively. When the workflow scheduler receives an arrival signal, the preprocessing phase will initialize decision parameters. Specifically, this phase calculates the original priority of each tasks based on the arrival time and the dependence relationships of the workflow, and allocates original subdeadlines for each tasks according to the worklow deadline. Afterwards, the preprocessing phase will deliver some entry tasks of the workflow to the resource allocation phase, and store other tasks into the workflow pool. The resource allocation phase takes charge of applying VMs from the resource manager. Then the resource allocation phase will also allocate the VMs to the tasks ready for scheduling. It is worth noting that only one task is allowed to wait on a VM in this phase every time. In this way, the fluctuation of task execution time on a VM can only affect the waiting task on the same VM. Besides, the feedback process will adjust the original priorities and subdeadlines of some relevant tasks in the workflow pool, when the workflow scheduler receives a completion signal. The feedback will also deliver the adjusted tasks to the resource allocation phase. Such that, the propagated impacts of the fluctuated task execution time can also be mitigated. Upon each received signal, each of the processing phases will be continuously triggered until all of the submitted workflows are processed successfully.

In the rest of this section, we present the three phases of the scheduling process in detail.

4.2 Preprocessing Phase

The settings of the initial decision parameters are the primary goals of the preprocessing phase, which involves determination of the original priority and subdeadline assignment of each task among arrival workflows.

For an arrival workflow, the early start time and the latest completion time of each task are important parameters for the scheduling decision. The former indicates the earliest time of a task to be executed, while the latter indicates the latest time of a task to be completed without missing the workflow deadline. They can be calculated by an assumption for scheduling all workflow

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

tasks on the VMs with the highest ranking. Let $M^{est}(t_i^s)$ denote the earliest start time of task t_i^s, which is defined as

$$M^{est}(t_i^s) = \begin{cases} a^s, & \text{if } t_i^s = t_{entry}^s, \\ \max\limits_{t_p^s \in pred(t_i^s)} & \{M^{est}(t_p^s) + w(\lambda_p^s) \\ & +d(e_{p,i}^s)\}, \text{otherwise}, \end{cases} \tag{8}$$

where $w(\lambda_p^s)$ represents the approximation weight of the stochastic execution time λ_p^s and can be calculated by Eq. (1). Accordingly, the earliest completion time of task t_i^s, denoted as $M^{eft}(t_i^s)$, can be calculated by

$$M^{eft}(t_i^s) = M^{est}(t_i^s) + w(\lambda_i^s). \tag{9}$$

We take the earliest completion time of each task as its original priority. Then, we use $M^{lct}(t_i^s)$ to denote the latest completion time of task t_i^s, which is defined as

$$M^{lct}(t_i^s) = \begin{cases} l^s, & \text{if } t_i^s = t_{exit}^s, \\ \min\limits_{t_q^s \in succ(t_i^s)} & \{M^{lct}(t_q^s) - w(\lambda_q^s) \\ & -d(e_{i,q}^s)\}, \text{otherwise}. \end{cases} \tag{10}$$

Although the latest completion time of each task is the bound of its completion time, the latest completion time is too slack. We use subdeadline to reduce the slack of the bound.

We utilize partial critical path (PCP) [9] to obtain all PCP paths of a workflow, and then transform the deadline of the workflow to the deadline of each PCP path. Therefore, the task scheduling on the PCP path becomes an end-to-end scheduling [34]. In this approach, the deadline of a PCP path can be divided into subdeadlines of the tasks on the path. The problem of matching the deadline of a workflow is transformed to the problem of matching the subdeadline of each task. The deadline of a PCP path is represented as the difference between the latest completion time of the last task t_k^s and the earliest start time of the first task t_1^s on the path, i.e., $M^{lct}(t_k^s) - M^{est}(t_1^s)$. We divide the path deadline into task subdeadlines on the PCP according to the proportion of the minimum execution time of the individual tasks [35]. In this way, the slack of individual tasks on the PCP is different but it's flexibility is the same, so that the subdeadline of task t_i^s can be described as

$$\begin{aligned} l_{sub}(t_i^s) = M^{est}(t_1^s) + \\ \frac{M^{eft}(t_i^s) - M^{est}(t_1^s)}{M^{eft}(t_k^s) - M^{est}(t_1^s)} \times (M^{lct}(t_k^s) - M^{est}(t_1^s)), \end{aligned} \tag{11}$$

where $M^{est}(t_1^s)$ is the execution time of the first task and $M^{lct}(t_k^s)$ is the execution time of the last task.

Additionally, we define $\delta(t_i^s)$ as the difference between $l_{sub}(t_i^s)$ and $M^{est}(t_i^s)$, i.e.,

$$\delta(t_i^s) = l_{sub}(t_i^s) - M^{est}(t_i^s), \tag{12}$$

which represents the maximum duration of task t_i^s without missing the workflow deadline.

4.3 Resource Allocation

The resource allocation phase stores the tasks ready for scheduling into a queue, with a non-descending order of the priority. In the next step, the tasks in the queue will in turn be scheduled on VMs, for which the resource allocation phase applies to the resource manager. In the following, we introduce some definitions and basic attributes for selecting VMs and scheduling tasks.

Definition 1. *Ready task: a task is an entry task, or its all necessary predecessors have been completed.*

For each ready task t_i^s to be scheduled on VM I_k, we need to estimate the time at which task t_i^s may start execution on VM I_k, and the time that VM I_k may take to execute task t_i^s. Thus, the predicted start time $\varpi(t_i^s)_k$ of task t_i^s on VM I_k can be calculated as

$$\max\{\Phi, \varphi(t_j^r)_k, \max_{t_p^s \in pred(t_i^s)} \{\zeta(t_p^s) + d(e_{p,i}^s)\}\}, \tag{13}$$

where Φ and $\varphi(t_j^r)_k$ indicate the current system time when scheduling for task t_i^s is incurred, and the predicted completion time of task t_j^r on VM I_k, respectively. Again, $\zeta(t_p^s)$ represents the completion time of task t_p^s on the VM executing it.

Theorem 1. *The predicted execution time of task t_i^s, denoted as $\psi(t_i^s)_k$, on VM I_k is equal to $B(I_k) \cdot w(\lambda_i^s)$.*

Proof. For the stochastic execution time λ_i^s of t_i^s with the normal distribution, $\lambda_i^s \sim N(\mu_i^s, {\sigma_i^s}^2)$, it can be proportionally multiplied by a constant a. Obviously, $a\lambda_i^s \sim N(a\mu_i^s, (|a|\sigma_i^s)^2)$. Then $B(I_k)\lambda_i^s \sim N(B(I_k)\mu_i^s, (B(I_k)\sigma_i^s)^2)$ can be obtained.

Thus, $B(I_k)\mu_i^s + B(I_k)\sigma_i^s = B(I_k)(\mu_i^s + \sigma_i^s) = B(I_k)w(\lambda_i^s)$. This proves the theorem. □

Accordingly, the predicted completion time $\varphi(t_i^s)_k$ of task t_i^s on VM I_k is calculated by

$$\varphi(t_i^s)_k = \varpi(t_i^s)_k + \psi(t_i^s)_k. \tag{14}$$

Definition 2. *Suitable VM: it is an active VM offered by the IaaS provider, which can complete the task to be scheduled on it before the task's subdeadline satisfying the following conditions:*

(1) The cost growth $\widetilde{cost}(t_i^s)_k$ of VM I_k for executing task t_i^s is the minimum. The definition of $\widetilde{cost}(t_i^s)_k$ is described as

$$\widetilde{cost}(t_i^s)_k = price(I_k) \times \psi(t_i^s)_k. \tag{15}$$

(2) When there are multiple VMs satisfying Condition (1), the one with the minimum idle time is selected because it is more likely to get higher resource utilization.

The selection of "suitable VM" is based on the heuristic method to minimize the rental cost of executing workflows and simultaneously improve the utilization efficiency of VMs as much as possible. According to the subdeadline of each task, the resource allocation phase firstly applies active VMs from the resource manager to satisfy Condition (1). If there are no active VMs satisfying Condition (1), it will apply leasing a new VM from the resource manager. Note that if an offered VM with the highest ranking of computing services fails to meet the subdeadline of the target task, the offered VM will still be leased by the resource manager to execute the task, because exceeding the task subdeadline in scheduling process for a specific task does not mean missing of the whole workflow deadline.

To improve the readability, the attributes related to the resource allocation phase are shown in Fig. 3. There are two VMs with the same ranking, (i.e., I_a and I_b) and three workflows (i.e., ξ_1,ξ_2 and ξ_3) in Fig. 3. Task t_1^1 is completed on VM I_a and task t_1^2 is executing on VM I_b at current system time Φ. Exactly at this moment, VM I_a begins executing task t_1^3 which had been waiting, while VM I_b was executing task t_1^2. Moreover, task t_2^1 becomes the ready task that needs to be immediately scheduled. Thus, we will calculate the predicted start time of task t_2^1 on I_a and I_b, respectively. For the former, the predicted completion time $\varphi(t_1^3)_a$

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

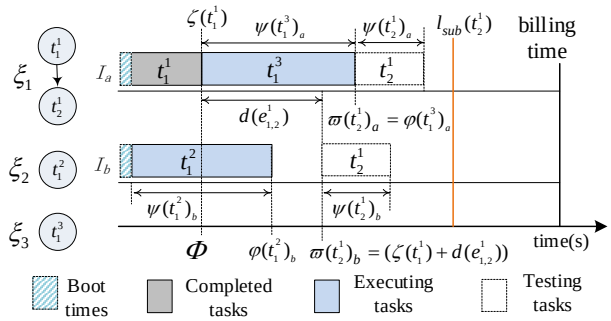


Fig. 3: The implications of basic attributes for scheduling.

of task t_1^3 is greater than the value of $\zeta(t_1^1) + d(e_{1,2}^1)$, so the value of $\varpi(t_2^1)_a$ is equal to $\varphi(t_1^3)_a$. For the latter, the predicted completion time $\varphi(t_1^2)_b$ of task t_1^2 is less than the value of $\zeta(t_1^1) + d(e_{1,2}^1)$. Hence, the value of $\varpi(t_2^1)_b$ is equal to $\zeta(t_1^1) + d(e_{1,2}^1)$. Consequently, VM I_a is the suitable VM because VM I_a is the one with the minimum idle time.

4.4 Feedback Processing

In NOSF, the immediate successors are likely to become ready tasks in case of the completion of a task on a VM. The feedback processing needs to determine whether these successors are ready or not. Moreover, the processing takes charge of adjusting the original priorities and subdeadlines of these ready tasks.

For a task that may be attached to multiple precedence constraints, it can be adjusted while all of its immediate predecessors have been completed. Thus, the adjustment of the earliest completion time of task t_i can be calculated as

$$\widehat{M^{eft}(t_i^s)} = \max_{t_p^s \in pred(t_i^s)} \{\zeta(t_p^s) + d(e_{p,i}^s) + w(\lambda_i^s)\}. \quad (16)$$

Accordingly, the adjustment of the earliest start time of task t_i^s is denoted as

$$\widehat{M^{est}(t_i^s)} = \widehat{M^{eft}(t_i^s)} - w(\lambda_i^s). \quad (17)$$

When a scheduling decision is made for task t_i^s, $\widehat{M^{eft}(t_i^s)}$ is regarded as task t_i^s's new priority for the upward rank.

Since our model calculates the sum of the expected value and the standard deviation to estimate the task execution time which is subject to the normal distribution, the completion of t_i^s's predecessor tasks may generate positive or negative deviations between the values of $\widehat{M^{est}(t_i^s)}$ and previous $M^{est}(t_i^s)$ due to normal distribution characteristics. Negative deviations may put task t_i^s's start time ahead, which does not violate the workflow deadline. On the contrary, positive deviations may delay its start time, which may miss the task subdeadline. But more importantly, the propagation of positive deviations may miss the whole deadline. Hence, the subdeadline of each task can be adjusted as

$$\begin{cases} \widehat{l_{sub}(t_i^s)} = \widehat{M^{est}(t_i^s)} + \delta(t_i^s), & \text{if } \widehat{M^{est}(t_i^s)} + \delta(t_i^s) \\ & < M^{lct}(t_i^s), \\ \widehat{l_{sub}(t_i^s)} = M^{lct}(t_i^s), & \text{otherwise.} \end{cases} \quad (18)$$

where if and only if $\widehat{M^{est}(t_i^s)} + \delta(t_i^s) < M^{lct}(t_i^s)$, the adjusted subdeadline of t_i^s will not exceed its $M^{lct}(t_i^s)$. Otherwise, the latest completion time of t_i^s will be delayed, compared to $M^{lct}(t_i^s)$, leading to delay propagation for successor tasks. As a result, $\widehat{l_{sub}(t_i^s)}$ is directly equal to $M^{lct}(t_i^s)$.

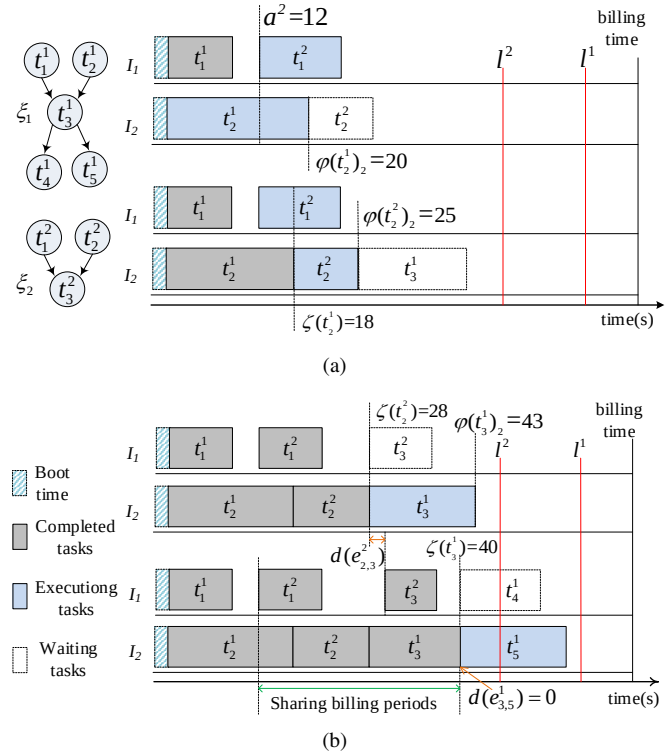


Fig. 4: The illustrative process for scheduling multiple workflows.

4.5 Illustrative Example of Scheduling Workflows

In this subsection, we introduce an example (shown in Fig. 4) to clearly illustrate how the scheduling process works in NOSF.

Assume that two workflows (i.e., ξ_1 and ξ_2) are submitted to the workflow scheduler at different time. For executing workflow ξ_1, moreover, the resource allocation phase leases two VMs (i.e., I_1 and I_2) with different rankings from the resource manager. We suppose that task t_1^1 and task t_2^1 have been scheduled on VM I_1 and VM I_2, respectively, as illustrated in the upper part of Fig. 4(a). When the time is 12s, I_1 has completed t_1^1 and is idle while I_2 is executing t_2^1. At this time, the workflow scheduler receives the arrival signal of ξ_2. The preprocessing phase initializes decision parameter for each tasks among ξ_2, immediately, and delivers t_1^2 and t_2^2 to the resource allocation phase. Assume that t_1^2 and t_2^2 are scheduled on I_1 and I_2, respectively. t_1^2 can be executed directly, while t_2^2 must wait due to the execution of t_2^1 on I_2. Afterwards, when I_2 completes t_2^1 at 18s, I_2 will directly execute t_2^2 on which was previously waiting, as shown in the lower part of Fig. 4(a). At this time, the workflow scheduler receives the completion signal of t_2^1. The feedback process starts to adjust the priority and subdeadline of t_3^1 and then delivers t_3^1 to the resource allocation phase, since it becomes the ready task. Suppose that t_3^1 is scheduled on I_2. Then, t_3^1 must wait due to the execution of t_2^2 at this moment. At time 28s, t_2^2 is completed, as revealed in the upper part of Fig. 4(b), so I_2 will immediately execute t_3^1. Meanwhile, the feedback process will update the priority and subdeadline of t_3^2, since the successor t_3^2 of t_2^2 becomes ready. Thus, t_3^2 will be schedule on a suitable VM. Suppose that I_1 is such a VM. In order to enable I_1 to execute t_3^2, I_2 need to transfer the output data of t_2^2 to I_1. This process need to take the time $d(e_{2,3}^2)$, and thus t_3^2 must wait. As time goes on, when t_3^1 is completed at time 40s, the feedback process will work again. As previously described, the

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

Algorithm 1: The preprocessing for an arrival workflow.

Input: A $D^s(T^s, E^s)$ corresponds to the arrival workflow ξ_s;
Output: A queue Q_r saving the entry tasks belonging to T^s; The workflow pool $pool$;

1 $Q_r \leftarrow \varnothing$;
2 Calculate $M^{est}(t_i^s)$, $M^{eft}(t_i^s)$ and $M^{lct}(t_i^s)$ for each task among T^s according to Eqs. (8), (9) and (10);
3 Traverse in turn all tasks among T^s from the exit nodes in order to find all PCP paths [9];
4 Traverse in turn all tasks on each PCP path from the first task on the path in order to compute $l_{sub}(t_i^s)$ and $\delta(t_i^s)$ for each task on the path according to Eqs. (11) and (12);
5 Add all the entry tasks in T^s into Q_r;
6 Add the other tasks in T^s into $pool$;
7 Call ScheduleTask(Q_r);

Algorithm 2: The feedback processing for the completion of a task on a VM.

Input: A completed task t_i^s; The VM I_k which completes task t_i^s; The workflow pool $pool$;
Output: A queue Q_r saving the tasks belonging to $succ(t_i^s)$ in the workflow pool;

1 $Q_r \leftarrow \varnothing$;
2 **if** *there exists a waiting task on* I_k **then**
3 Execute the waiting task on I_k;
4 **for** *each* $t_q^s \in succ(t_i^s)$ **do**
5 **if** *all of* $pred(t_q^s)$ *have been completed* **then**
6 Calculate $\widehat{M^{eft}(t_q^s)}$, $\widehat{M^{est}(t_q^s)}$ and $\widehat{l_{sub}(t_q^s)}$ for t_q^s according to Eqs. (16), (17) and (18);
7 Add task t_q^s into Q_r and remove task t_q^s from $pool$;
8 Call ScheduleTask(Q_r);

successors t_4^1 and t_5^1 of t_3^1, which become ready, will be scheduled on the suitable VM for executing. At this point, I_1 and I_2 are idle. We assume that t_4^1 and t_5^1 are scheduled on I_1 and I_2, respectively. Then, I_2 can immediately execute t_5^1, due to $d(e_{3,5}^1)$=0, but I_1 will execute t_4^1 after receiving the output data of t_3^1, as shown in the lower part of Fig. 4(b).

From Fig. 4, it can be observed that the actual task completion time is different from the predicted task completion time. Especially, $\zeta(t_2^1)$ and $\zeta(t_3^1)$ is less than $\varphi(t_2^1)_2$ and $\varphi(t_3^1)_2$, respectively, whereas $\zeta(t_2^2)$ is greater than $\varphi(t_2^2)_2$. As previously mentioned, there exists a gap between the estimated task execution time and the actual task execution time on a VM, due to the fluctuated VM performance. However, in NOSF, the fluctuation of task execution time on a VM can only affect the waiting task, such as t_2^2 and t_3^1. Besides, tasks among workflows submitted during the same periods are orchestrated to share the billing time, as shown in the lower part of Fig. 4(b).

5 Deadline-Aware Heuristic Scheduling Algorithm

In this section, we propose a deadline-aware heuristic scheduling algorithm based on NOSF, and analyze the time complexity of the algorithm.

The proposed algorithm consists of two steps, including the preprocessing for arrival workflows shown in Alg. 1 and the feedback processing for the completion of a task on a VM shown in Alg. 2. When NOSF receives the arrival signal of a new workflow, the preprocessing phase (i.e., Alg. 1) is triggered immediately. Firstly, the algorithm calculates the M^{est}, M^{eft} and M^{lct} of all tasks among the arrival workflow according to Eqs.(8)-(10) (Line 2). Then, in turn, the algorithm traverses all tasks among the workflow from the exit nodes according to the dependencies between tasks, and finds out all PCPs (Line 3). The details of traversing tasks to find PCPs can be referred to [9]. Based on the obtained PCPs, the algorithm traverses all tasks on each PCP to assign the subdeadlines according to Eqs. (11) and (12) (Line 4). Whereafter, the algorithm fetches the entry tasks among the workflow into queue Q_r (Line 5), and then takes all other remaining tasks of the workflow into the workflow pool (Line 6). Finally, the algorithm calls the Function ScheduleTask through taking queue Q_r as the input (Line 7).

When NOSF receives a completion signal of a VM, it will trigger the feedback process as shown in Alg. 2. Once VM I_k completes task t_i^s, the waiting task will be executed directly (Lines 2 and 3) if there is a waiting task on I_k. Subsequently, the algorithm will check whether the immediate successors, restrained by t_i^s, are the ready tasks. If the successors are ready, the algorithm will adjust the corresponding M^{eft}s, M^{est}s, and l_{sub}s of the ready tasks. Moreover, It will fetch the ready tasks into queue Q_r and remove them from the workflow pool (Lines 4-7). Finally, the algorithm calls the Function ScheduleTask in terms of queue Q_r (Line 8).

Alg. 3 is in charge of scheduling the tasks in the queue in which the ready tasks are passed by Alg. 1 and Alg. 2 on suitable VMs. The algorithm sorts queue Q_r by a non-descending order of the priority (Line 1). Notice that if task t_i^s belongs to a new workflow in queue Q_r, then the initial $M^{est}(t_i^s)$ of t_i^s is regarded as the priority. Else, the value of $\widehat{M^{est}(t_i^s)}$ is indicated as the priority. Then the algorithm traverses each task t_i^s in queue Q_r in order to seek a set $\mathbb{I}$ of candidate VMs for task t_i^s (Lines 2-11). Although all immediate predecessors of task t_i^s in queue Q_r have been completed at this moment, the output data from the predecessors may be transferred. We need to estimate the largest arrival time of the output data from immediate predecessors of task t_i^s by trying to test the candidates for suitable VMs. Thus, if there exists an idle VM I_k, Line 6 calculates the predicted start time $\varpi(t_i^s)_k$ of task t_i^s to be executed on VM I_k. Otherwise, the predicted start time $\varpi(t_i^s)_k$ of task t_i^s is calculated by Line 8. Moreover, Line 9 calculates $\varphi(t_i^s)_k$ and $cost(t_i^s)_k$ of task t_i^s on the basis of Eqs. (14) and (15). Next, the algorithm decides whether VM I_k is a suitable candidate VM for task t_i^s (Line 10). If VM I_k is a candidate, it will be added into set $\mathbb{I}$ (Line 11). Again, the obtained candidate set $\mathbb{I}$ may include multiple VMs, the algorithm will find out a VM with the minimal idle time from $\mathbb{I}$ as the suitable VM for t_i^s (Line 13). Finally, if each of active VMs does not match the requirement for executing task t_i^s, the algorithm will lease a new suitable VM from all VM rankings the cloud provider offers (Lines 14-15). If all VM rankings, nevertheless, can not satisfy the subdeadline of task t_i^s, a new VM with the highest ranking will be leased (Line 17).

Theorem 2. *The time complexity for processing an arrival workflow ξ_s with Alg.1 is $O(|T|^2 + |N_{in}|log(|N_{in}|) + |N_{in}|^2|N_{vm}|)$,*

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

Algorithm 3: Function ScheduleTask(Q_r)

Input: A queue Q_r saving the ready tasks;
1 Sort Q_r by a non-descending order of the priority;
2 **for** *each task* $t_i^s \in Q_r$ **do**
3 $Growth \leftarrow +\infty, \mathbb{I} \leftarrow \varnothing$;
4 **for** *each active* I_k **do**
5 **if** I_k *is idle* **then**
6 $\varpi(t_i^s)_k \leftarrow \max\{\Phi, \max_{t_p^s \in pred(t_i^s)} \{\zeta(t_p^s) + d(e_{p,i}^s)\}\}$;
7 **else**
8 $\varpi(t_i^s)_k \leftarrow \max\{\varphi(t_j^r)_k, \max_{t_p^s \in pred(t_i^s)} \{\zeta(t_p^s) + d(e_{p,i}^s)\}\}$;
9 Calculate $\varphi(t_i^s)_k$ and $\widetilde{cost}(t_i^s)_k$ for t_i^s according to Eqs. (14) and (15);
10 **if** $\varphi(t_i^s)_k \leq \widehat{l_{sub}(t_i^s)}$ *and* $\widetilde{cost}(t_i^s)_k < Growth$ **then**
11 Add I_k into $\mathbb{I}$, then $Growth \leftarrow \widetilde{cost}(t_i^s)_k$;
12 **if** $\mathbb{I}$ *is not* $\varnothing$ **then**
13 Find out the VM with the minimal idle time in $\mathbb{I}$, then schedule t_i^s on it;
14 **else if** *There exists VM* I_h *with the minimal* $\widetilde{cost}(t_i^s)_h$, *which executes task* t_i^s *while satisfying* $\varphi(t_i^s) \leq \widehat{l_{sub}(t_i^s)}$ **then**
15 Lease a new VM I_h with the minimal $\widetilde{cost}(t_i^s)_h$;
16 **else**
17 Lease a new VM with the highest ranking of computing services;
18 Schedule t_i^s on the new VM after it has been initiated;

where $|T|$ is the number of tasks among workflow ξ_s, and $|N_{in}|$ is the maximum indegree of a task among ξ_s, and $|N_{vm}|$ represents the number of active VMs, respectively.

Proof. Processing for an arrival workflow consists of two parts. The first part defines preprocessing the new workflow. The second part is described in Alg. 3 for scheduling entry tasks in queue Q_r. We assume that a $D^s(T^s, E^s)$ corresponding to workflow ξ_s consists of $|T|$ tasks and $|E|$ edges. As D^s is a DAG, the maximum number of $|E|$ is $\frac{(|T|-1)(|T|-2)}{2}$, we suppose that $|E| \simeq O(|T|^2)$. Line 2 in Alg. 1 is to calculate M^{est}, M^{eft} and M^{lct} for each task, which needs time $O(|T|^2)$. The time complexity of the PCP algorithm (Line 3 in Alg. 1) is $O(|T|^2)$ [9]. The subdeadline assignment requires time $O(|T|)$. Hence, the time complexity of the first part is $O(|T|^2)$.

Subsequently, the length of queue Q_r delivered to Alg. 3 is at most equal to $|N_{in}|$. In Alg. 3, sorting tasks in queue Q_r needs time $O(|N_{in}|log(|N_{in}|))$. For each task t_i^s, Alg. 3 needs to traverse all active VMs and VM rankings for scheduling. In the part of traversing active VMs, Alg. 3 needs to try each of immediate predecessors of t_i^s for calculating the value of $\varpi(t_i^s)$. The calculation requires time $O(|N_{in}|)$. Thus, the time complexity of traversing active VMs is $O(|N_{vm}||N_{in}|)$. Additionally, Lines14-15 of Alg. 3 need to traverse all VM rankings for leasing a new VM. Since the number of VM rankings is far less than that of active VMs, the time required traversing VM rankings can be regarded as a constant. As the length of queue Q_r is $|N_{in}|$, scheduling each task in queue Q_r needs time $O(|N_{in}|log(|N_{in}|) + |N_{in}|^2|N_{vm}|)$, correspondingly. Thus, the time complexity for scheduling a new submitted workflow with Alg.1 is $O(|T|^2 + |N_{in}|log(|N_{in}|) + |N_{in}|^2|N_{vm}|)$. This proves the theorem. □

Theorem 3. *The time complexity for the feedback process with Alg. 2 is $O(|N_{out}|log(|N_{out}|) + |N_{out}||N_{in}||N_{vm}|)$, where $|N_{out}|$ is the maximum outdegree of a task among ξ_s.*

Proof. When Alg. 2 adjusts the immediate successors of a completed task t_i^s, it needs to traverse each of $succ(t_i^s)$, which needs time $O(|N_{out}|)$. For each immediate successors of task t_i^s, again, the algorithm needs to determine whether their immediate predecessors have been completed for the purpose of adjusting, which requires time $O(|N_{in}|)$. Thus, the time complexity of adjusting the immediate successors of task t_i^s in the algorithm is $O(|N_{out}||N_{in}|)$. Besides, in Alg. 2, the length of queue Q_r saving the ready tasks is $|N_{out}|$, which will be delivered to Alg. 3. Consequently, the time complexity of Alg. 3 is $O(|N_{out}|log(|N_{out}|) + |N_{out}||N_{in}||N_{vm}|)$ according to Proof for Theorem 2. Hence, The time complexity for the feedback process is $O(|N_{out}|log(|N_{out}|) + |N_{out}||N_{in}||N_{vm}|)$. This proves the theorem. □

6 Performance Evaluation

To evaluate the performance of different algorithms, we compare NOSF with ROSA [25] and IC-PCPD2 [9] algorithms, which are two of the existing workflow scheduling algorithms in clouds.

ROSA is an online scheduling algorithm for multiple workflows, considering the stochasticity of task execution time. Moreover, ROSA utilizes the quantile for the normal distribution to estimate the stochastic task execution time. This algorithm ranks the tasks ready for scheduling based on their predicted latest start time, and take tasks' predicted latest completion time as decision parameters. Then, these tasks are in turn scheduled according to the setting priorities and decision parameters.

IC-PCPD2 is a scheduling algorithm for a single workflow under the deterministic task execution time, and adopts a similar method of deadline assignment with ours as well. This algorithm will in turn schedule the tasks of the workflow in a breadth-firth way, while satisfying the subdeadline. In order to enable IC-PCPD2 to be applicable for the online arrvial of workflows, we trigger it to schedule all the workflow tasks as soon as a workflow arrives.

6.1 Experimental Workflows and Performance Metrics

We evaluate the three algorithms based on five realistic workflow datasets that are available in DAX [2], including Montage, CyberShak, Epigenomics, LIGO and SIPHT. We create two workflow template sets, involving a medium set and a large set, based on the workflow applications. The medium template set comprises 30, 50 and 100 tasks, while the large template set is composed of these workflows containing 1000 tasks. Accordingly, there are 15 workflow templates in the former, and there are 5 workflow templates in the latter.

Our optimization objective is not only to minimize the total cost of VM leasing, but also to utilize two metrics to evaluate the performance of scheduling algorithms, which are presented below.

(1) Resource Utilization

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

We use the average utilization efficiency of VMs to evaluate the three algorithms, which is defined as

$$Utilization = \sum_{k=1}^{|\mathbb{I}|} ET(I_k) \Big/ \sum_{k=1}^{|\mathbb{I}|} AT(I_k), \tag{19}$$

where $ET(I_k)$ and $AT(I_k)$ represent the processing time and the total active time (including the processing and idle time) of VM I_k during executing the workflow set Ω.

(2) Deadline Violation Probability

We define two metrics related to deadline violation shown in Eqs. (20) [10] and (21), respectively, to evaluate the three algorithms.

$$TimeViolation = \sum_{s=1}^{|\Omega|} \frac{MakeSpan_s + a^s - l^s}{l^s - a^s} \Big/ |\Omega|, \tag{20}$$

$$CountViolation = \overline{|\Omega|} \Big/ |\Omega|, \tag{21}$$

where $|\Omega|$ represents the total number of workflow set Ω and $\overline{|\Omega|}$ represents the number of missed workflow deadlines during processing workflow set Ω. Eq. (20) indicates the average duration percentage of exceeding workflow deadlines in workflow set Ω. And Eq. (21) indicates the average percentage of the number of missed workflow deadlines in Ω.

TABLE 2: VM Parameters Used in Experiments

VM Type	vCPU	Price per time unit($)	$B(\cdot)$[1]
m2.4xlarge	8	0.98	1.0
m2.2xlarge	4	0.49	1.2
m1.xlarge	4	0.35	1.3
m2.xlarge	2	0.245	1.4
m1.large	2	0.175	1.6
m1.medium	1	0.087	1.8
m1.small	1	0.044	2.0

[1] $B(\cdot)$ indicates the processing time weight of the corresponding VM ranking.

6.2 Simulation Settings

We use a laptop equipped with 3.3GHz Intel Core i5 processor and 8GB memory as the experimental platform, and implement a discrete time simulator for cloud workflow scheduling simulation by using JAVA. In the experiments, VM parameters are configured on the basis of the General Purpose instance group that Amazon EC2 provides by on-demand purchasing in US East region. The configurations [13] and prices for VMs are listed in Table 2. We assume that without any limitations, the customer can lease VMs with any rankings, and all required softwares for executing workflows can be installed on each leased VM. Also, we set the average bandwidth between VMs to 100Mbps and the billing period to 3600 seconds, as well as the boot time of a VM to 97 seconds, based on the results of Amazon EC2.

To simulate the random arrival of workflows, we select a workflow with a stochastic time interval following the Poisson distribution with $\lambda = 0.01$, from one of the above two template sets to submit to the scheduler. In addition, the task runtime, which is recorded in the workflow traces, is regard as the baseline of the task execution time. Then, the baseline execution time of task t_i^s is denoted as $f_t(t_i^s)$. As the execution time of t_i^s is modelled as a stochastic variable following the normal distribution with the expected value $\mu_{t_i^s}$ and the variance $\sigma^2_{t_i^s}$, Eq (22) is defined as describing the stochastic characteristic:

$$\mu_{t_i^s} = f_t(t_i^s), \quad \sigma_{t_i^s} = f_t(t_i^s) \cdot \alpha, \tag{22}$$

where α is a variance factor that indicates the fluctuation of the task execution time. In our experiments, the stochastic execution time of task t_i^s is defined using a normal random number generator with the expected value of $\mu_{t_i^s}$ and the standard deviation of $\sigma_{t_i^s}$.

To assign a deadline to each workflow, we need to define the makespan for the shortest schedule M_S, which is gained by scheduling each workflow task on a distinct VM with the highest ranking while all data transfer time is regarded as zero. Therefore, the deadline of workflow ξ_s can be calculated as $l^s = a^s + \beta \cdot M_S$, where β is a deadline factor.

The performance of scheduling algorithms is affected by multiple parameters, such as the variance factor α of the task execution time, the deadline factor β, as well as the count of tasks among the workflows. Details of these parameters are listed as follows:

(1) The variance factor α is tested with the values $\{0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5\}$, where the default value of α is $\{0.2\}$.

(2) The deadline factor β is tested with the values $\{2, 2.5, 3, 3.5, 4\}$, where the default value of β is $\{2\}$.

(3) The number of workflows from the medium template set is tested with the value $\{1000\}$, and that from the large template set is tested with the value $\{200\}$.

We utilize randomly selected workflows to test each algorithm. For each group of parameter settings, each algorithm runs independently for 30 times, then the minimum, average and maximum of the results are obtained according to Eqs. (3), (19), (20) and (21).

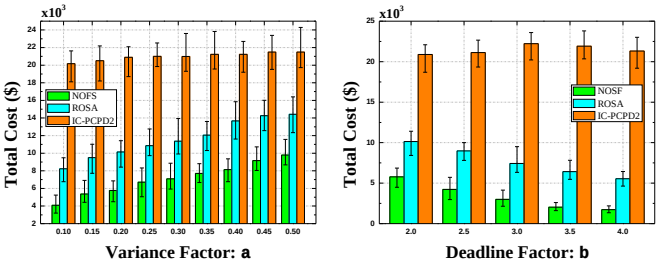
6.3 Evaluation Results

6.3.1 Total VM Rental Cost and Resource Utilization

From the experimental results obtained by the medium template set, we can observe the total VM rental cost and the utilization efficiency of leasing VMs in the three algorithms. According to Figs. 5(a) and 5(b), the total cost obtained by NOSF is less than the cost of ROSA and IC-PCPD2 when α and β increase. Moreover, Figs. 6(a) and 6(b) reveal that the utilization efficiency of VMs achieved by NOSF are still better than the utilization efficiency of ROSA and IC-PCPD2 when α and β increase. There are two major reasons for this. Firstly, NOSF adjusts the predicted start time and the priorities of the ready tasks in each step, which prohibits the propagation of gaps between the predicted and real completion time of tasks. Secondly, NOSF seeks heuristically the suitable VMs for the ready tasks. Therefore, the idle time slots on each VM may be impacted as possible. For example, the last idle time in the billing period may be utilized sufficiently. Since ROSA does not control the propagation of such gaps, it consequently results in a lower utilization efficiency. IC-PCPD2 is an off-line algorithm designed for a single workflow. Inevitably, there exist a huge number of idle time slots on leased VMs. Additionally, IC-PCPD2 does not apply any strategy to handle the influence of the fluctuated execution time, which may also result in unpredicted idle time slots and poor utilization efficiency.

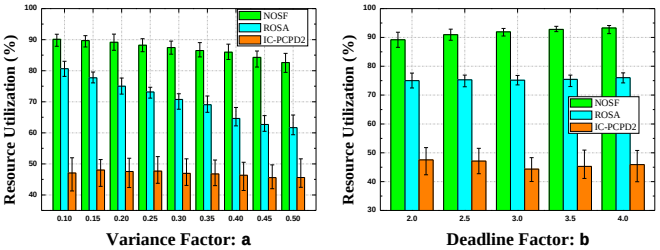
2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing



(a) (b)

Fig. 5: The total cost of scheduling workflows.



(a) (b)

Fig. 6: The resource utilization of scheduling workflows.

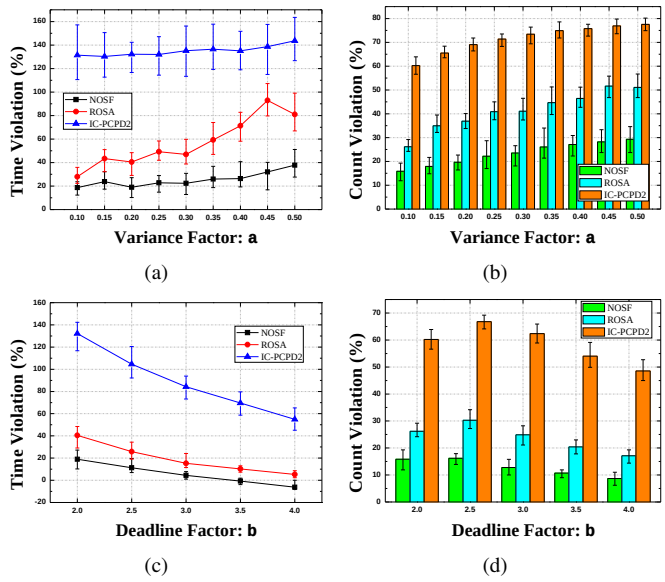
6.3.2 Deadline Violation Probability

In this subsection, we evaluate the deadline violation probability of the three algorithms for scheduling a medium template set. Fig. 7 shows that all three algorithms have a few workflows that do not complete before the deadlines under fluctuated task execution time. We can see that the deadline violation probability of IC-PCPD2 is much higher than the results of NOSF and ROSA because IC-PCPD2 fails to deal with the influence of the fluctuated execution time. Specifically, from Figs. 7(a) and 7(b), we can observe that NOSF is obviously less than ROSA, in terms of the average duration of exceeding the deadlines and the number of workflows unsatisfying the deadlines, when α increases. This can be interpreted that the feedback process in NOSF controls the accumulation of the fluctuated task execution time for mitigating its propagation, so that the deadline violation probability is reduced. Figs. 7(c) and 7(d) reveal that the deadline violation probabilities of the three algorithms decrease with the increment of β. This demonstrates that prolongation of deadline enlarges the slack of tasks, so that each of the algorithms can schedule more workflows satisfying the relevant deadlines.

6.3.3 ime Efficiency

In order to evaluate the time efficiency of the three algorithms, in this subsection, we compare the average execution time of the algorithms under various α and β.

From Fig. 8, we can observe that the time overhead of NOSF is smaller than that of IC-PCPD2 but higher than that of ROSA. This is because NOSF will adjust the predicted start time and the priorities of the ready tasks, then calculate theirs subdeadlines when a task is completed. It results in more execution time. In addition, Fig. 8(a) shows that the execution time of NOSF gradually decreases while the execution time of ROSA slightly increases with the increase of α. The differences between the two algorithms get smaller as α increases. From Fig. 8(b), we can also observe the similar tend. Besides, IC-PCPD2 has to release more

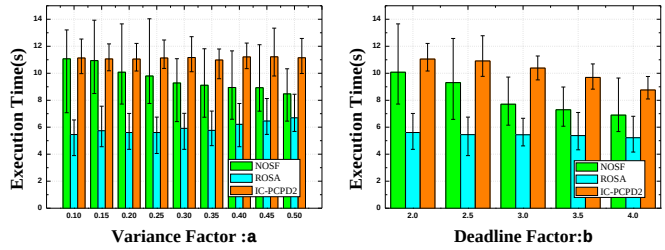


(a) (b)

(c) (d)

Fig. 7: Deadline violation of scheduling workflows.

VMs due to its inefficient resource utilization, which leads to more time consumption for seeking VMs for each task.



(a) (b)

Fig. 8: The average execution time of scheduling algorithms.

6.3.4 Performance of Scheduling Large Workflows

In this subsection, we evaluate the experimental results of the three algorithms for scheduling workflows from the large template set.

As shown in Fig. 9, we can obviously see that the performance of NOSF for scheduling large workflows is still much better than IC-PCPD2 and ROSA in terms of the total VM rental cost and the utilization efficiency of leasing VMs, as well as the deadline violation probability. Besides, Fig. 9(a) shows that the total VM rental cost of each algorithm for scheduling large workflows is much higher than that for scheduling workflows from the medium template set. It is obvious that the more tasks among a workflow need longer time that leased VMs take to execute them, which results in higher costs. Moreover, Fig. 9(b) indicates that the number of tasks in workflows has little impact on resource utilization of scheduling multiple workflows. Additionally, Fig. 9(c) shows that the number of missed workflow deadlines is much higher in each of the algorithms for scheduling large workflows than that for scheduling workflows from the medium template set. However, Fig. 9(d) reveals that the average duration of exceeding the deadlines are much lower in each algorithm for scheduling large workflows for the following reasons. On one hand, the execution of the workflow containing more tasks results in the

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

more serious accumulation of the fluctuated task execution time, so that the number of missed workflow deadlines is higher. On the other hand, the makespan of the workflow is longer, due to containing more tasks. The proportion of the length of time taken to execute the workflow in the whole makespan is reduced after missing the deadline in each algorithm.

To sum up, the above experiment results demonstrate that the performance of NOSF is better than both the IC-PCPD2 and ROSA algorithms. Nevertheless, we only consider that the cost growth of an active VM must be the minimum, which is currently active before scheduling a new task on it regardless of the remaining billing period, when we make the selection decision for a suitable VM. This may lead to myopic optimization, due to lack of information about the remaining billing period.

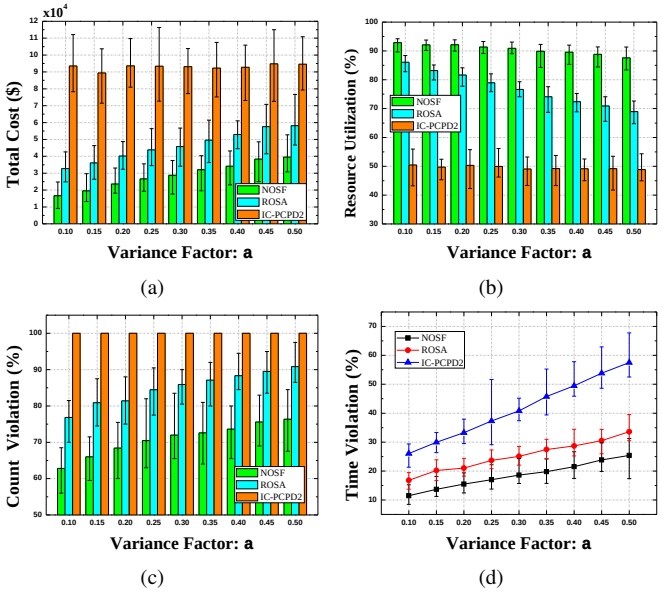


Fig. 9: Performance scheduling of large workflows.

6.4 Discussion on Estimation of Task Execution Time

We evaluate the impact of task execution time estimation on the performance of NOSF and ROSA algorithms in this subsection. We use $\mu + \eta \cdot \sigma$ to estimate the task execution time, where η is a factor varying from 0 to 3 with an increment of 0.25. For the sake of fairness, each of the two algorithms utilizes the expression $\mu+\eta\cdot\sigma$ for estimation, named as NOSF* and ROSA*, respectively. We test the performance of the two algorithms, afterwards, for scheduling the first workflow template set.

Fig. 10 shows that the performance of NOSF* is superior to that of ROSA*. It also verifies that the feedback process and the adjustment mechanism in NOSF* work properly. More remarkable, the experimental results of Fig. 10 show that NOSF* and ROSA* change with the same trend in terms of VM rental cost, deadline violation probability and resource utilization efficiency, as η increases. According to Fig. 10(a), particularly, we can observe that the total VM rental cost obtained by NOSF* and ROSA* are considerably high when $\eta = 0$. Next, the total VM rental cost obtained by the two algorithms decreases obviously when η increases ($\eta \leq 1.5$). While η is around 1.5, NOSF* and ROSA* obtain the lowest approximate cost, without the significant variation. Additionally, in respect of resource utilization, Fig. 10(b) shows that the utilization efficiency of VMs in NOSF* and ROSA* are decreased visibly as $\eta = 0$, compared to their original algorithms. Then, the utilization efficiency of VMs in NOSF* and ROSA* improves, with the increase of η ($\eta \leq 1.25$). When $1.25 \leq \eta \leq 1.75$, the variation of utilization efficiency is tiny in the two algorithms and both the two algorithms can achieve a better utilization efficiency. Again, Figs. 10(c) and 10(d) show that the deadline violation probability of NOSF* and ROSA* decreases sharply when η varies from 0 to 3. We can also observe that the deadline violation probability of both the two algorithms almost reaches a lower value when η is around 3. This is because a normal stochastic variable located in the range of $[\mu - 3 \cdot \sigma, \mu + 3 \cdot \sigma]$, with the probability of 99.74%. This result accounts for the fact that a vast majority of values of task execution time are located in the estimation range of $\mu+3\cdot\sigma$ when the expression $\mu+3\cdot\sigma$ is used to estimate the task execution time. In this case, the fluctuation of the task execution time almost does not exceed the range. Hence, the two algorithms can process even more workflows before the deadlines.

The results in this group of experiments demonstrate that the estimation value of the task execution time with stochastic characteristic acutely effects the performance of NOSF* and ROSA* algorithms. The estimation of the stochastic execution time with the normal distribution enables the scheduling algorithms to achieve a lower VM rental cost and a better utilization efficiency of VMs. However, in IaaS clouds, we can still optimize the rental cost and the utilization efficiency of VMs further by setting an appropriate value of η. It is worth noting that our solution is designed by neglecting many factors, such as the energy consumption and the resource supply capacities of the providers, VM delay, etc. These factors, in fact, have some influence on the performance of scheduling algorithms, which should be carefully considered in our future work for further improving the performance of online workflow scheduling.

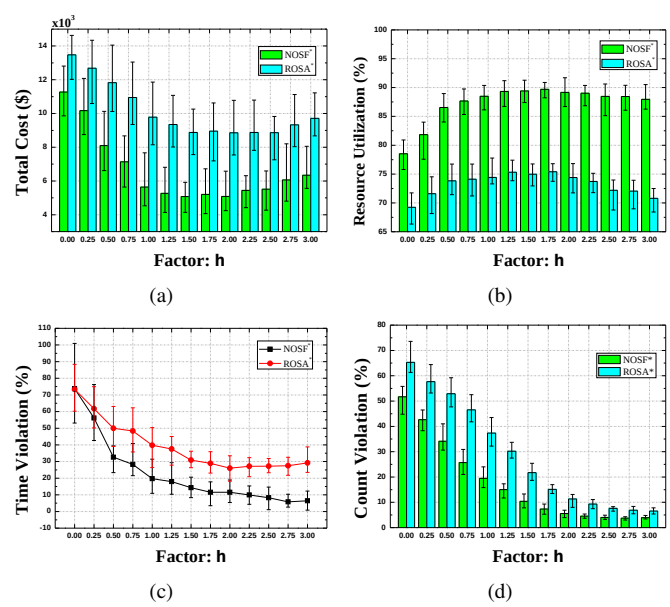


Fig. 10: The impact of task execution time estimation.

7 Conclusions and Future Work

In this paper, we have investigated the online workflow scheduling problem that exists in IaaS clouds, where VMs have fluctuating

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

performance causing uncertain task execution time and workflows arrive continuously and stochastically. To capture the stochasticity, we have proposed NOSF, an online multi-workflow scheduling framework, to schedule the deadline-constrained workflows in IaaS clouds online. Moreover, we have developed a deadline-aware workflow scheduling algorithm, based on NOSF, to simultaneously minimize the total VM rental cost and deadline violation probability of workflows. Extensive simulation results on realistic workflow datasets have demonstrated that the proposed algorithm outperforms two existing workflow scheduling solutions (i.e., IC-PCPD2 and ROSA), in terms of reducing the VM rental cost and the deadline violation probability, as well as improving the resource utilization efficiency. In our future work, we aim to investigate the joint optimization of the VM rental cost and energy efficiency for multi-workflow scheduling in IaaS clouds, considering the different characteristics of heterogeneous VMs and the limited number of VMs with fluctuated performance.

Acknowledgments

This work is supported by the Natural Science Foundation of China (No. 61702562), the 111 Project (No. B18059), the International Science & Technology Cooperation Program of China (No. 2013DFB10070) and China Hunan Provincial Science & Technology Program (No. 2012GK4106).

References

[1] Y. Zhang, J. Ren, J. Liu, C. Xu, H. Guo, and Y. Liu, "A survey on emerging computing paradigms for big data," *Chinese Journal of Electronics*, vol. 26, no. 1, pp. 1–12, 2017.

[2] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," *Future Generation Computer Systems*, vol. 29, no. 3, pp. 682–692, 2013.

[3] J. Ren, H. Guo, C. Xu, and Y. Zhang, "Serving at the edge: A scalable iot architecture based on transparent computing," *IEEE Network*, vol. 31, no. 5, pp. 96–105, 2017.

[4] L. T. Yang and M. Guo, *High-performance computing: paradigm and infrastructure*. John Wiley & Sons, 2005, vol. 44.

[5] V. K. Naik, C. Liu, L. Yang, and J. Wagner, "Online resource matching for heterogeneous grid environments," in *ccgrid*. IEEE, 2005, pp. 607–614.

[6] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Computing Surveys (CSUR)*, vol. 31, no. 4, pp. 406–471, 1999.

[7] D. P. Vidyarthi, B. K. Sarker, A. K. Tripathi, and L. T. Yang, *Scheduling in distributed computing systems: Analysis, design and models*. Springer Science & Business Media, 2008.

[8] M. Lin and L. T. Yang, "Hybrid genetic algorithms for scheduling partially ordered tasks in a multi-processor environment," in *Real-Time Computing Systems and Applications, 1999. RTCSA'99. Sixth International Conference on*. IEEE, 1999, pp. 382–387.

[9] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Deadline-constrained workflow scheduling algorithms for infrastructure as a service clouds," *Future Generation Computer Systems*, vol. 29, no. 1, pp. 158–169, 2013.

[10] Z. Cai, X. Li, R. Ruiz, and Q. Li, "A delay-based dynamic scheduling algorithm for bag-of-task workflows with stochastic task execution times in clouds," *Future Generation Computer Systems*, vol. 71, pp. 57–72, 2017.

[11] Z. Cai, X. Li, and R. Ruiz, "Resource provisioning for task-batch based workflows with deadlines in public clouds," *IEEE Transactions on Cloud Computing*, 2017.

[12] R. N. Calheiros and R. Buyya, "Meeting deadlines of scientific workflows in public clouds with tasks replication," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 7, pp. 1787–1796, 2014.

[13] H. Chen, X. Zhu, D. Qiu, L. Liu, and Z. Du, "Scheduling for workflows with security-sensitive intermediate data by selective tasks duplication in clouds," *IEEE Transactions on Parallel and Distributed Systems*, 2017.

[14] B. Lin, W. Guo, N. Xiong, G. Chen, A. V. Vasilakos, and H. Zhang, "A pretreatment workflow scheduling approach for big data applications in multicloud environments," *IEEE Transactions on Network and Service Management*, vol. 13, no. 3, pp. 581–594, 2016.

[15] B. P. Rimal and M. Maier, "Workflow scheduling in multi-tenant cloud computing environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 1, pp. 290–304, 2017.

[16] J. Schad, J. Dittrich, and J.-A. Quiané-Ruiz, "Runtime measurements in the cloud: observing, analyzing, and reducing variance," *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 460–471, 2010.

[17] D. Bruneo, "A stochastic model to investigate data center performance and qos in iaas cloud computing systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, pp. 560–569, 2014.

[18] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioningand scheduling algorithm for scientific workflows on clouds," *IEEE Transactions on Cloud Computing*, vol. 2, no. 2, pp. 222–235, 2014.

[19] F. Wu, Q. Wu, Y. Tan, R. Li, and W. Wang, ": Partial critical path budget balanced scheduling algorithms for scientific workflow applications," *Future Generation Computer Systems*, vol. 60, pp. 22–34, 2016.

[20] V. Arabnejad, K. Bubendorfer, and B. Ng, "Scheduling deadline constrained scientific workflows on dynamically provisioned cloud resources," *Future Generation Computer Systems*, 2017.

[21] M. A. Rodriguez and R. Buyya, "Budget-driven scheduling of scientific workflows in iaas clouds with fine-grained billing periods," *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, vol. 12, no. 2, p. 5, 2017.

[22] J. Sahni and D. Vidyarthi, "A cost-effective deadline-constrained dynamic scheduling algorithm for scientific workflows in a cloud environment," *IEEE Transactions on Cloud Computing*, 2015.

[23] Y.-R. Wang, K.-C. Huang, and F.-J. Wang, "Scheduling online mixed-parallel workflows of rigid tasks in heterogeneous multi-cluster environments," *Future Generation Computer Systems*, vol. 60, pp. 35–47, 2016.

[24] H. Chen, X. Zhu, D. Qiu, and L. Liu, "Uncertainty-aware real-time workflow scheduling in the cloud," in *Cloud Computing (CLOUD), 2016 IEEE 9th International Conference on*. IEEE, 2016, pp. 577–584.

[25] H. Chen, X. Zhu, G. Liu, and W. Pedrycz, "Uncertainty-aware online scheduling for real-time workflows in cloud service environment," *IEEE Transactions on Services Computing*, 2018.

[26] X. Peng, J. Ren, L. She, D. Zhang, J. Li, and Y. Zhang, "Boat: A block-streaming app execution scheme for lightweight iot devices," *IEEE Internet of Things Journal*, vol. 5, no. 3, pp. 1816–1829, 2018.

[27] D. Zhang, Z. Chen, L. X. Cai, H. Zhou, S. Duan, J. Ren, X. Shen, and Y. Zhang, "Resource allocation for green cloud radio access networks with hybrid energy supplies," *IEEE Transactions on Vehicular Technology*, vol. 67, no. 2, pp. 1684–1697, 2018.

[28] K. Li, X. Tang, B. Veeravalli, and K. Li, "Scheduling precedence constrained stochastic tasks on heterogeneous cluster systems," *IEEE Transactions on computers*, vol. 64, no. 1, pp. 191–204, 2015.

[29] L.-C. Canon and E. Jeannot, "Correlation-aware heuristics for evaluating the distribution of the longest path length of a dag with random weights," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 11, pp. 3158–3171, 2016.

[30] S. Jang, X. Wu, V. Taylor, G. Mehta, K. Vahi, and E. Deelman, "Using performance prediction to allocate grid resources," *Texas A&M University, College Station, TX, GriPhyN Technical Report*, vol. 25, 2004.

[31] M. A. Iverson, F. Ozguner, and L. C. Potter, "Statistical prediction of task execution times through analytic benchmarking for scheduling in a heterogeneous environment," in *Heterogeneous Computing Workshop, 1999.(HCW'99) Proceedings. Eighth*. IEEE, 1999, pp. 99–111.

[32] G. R. Nudd, D. J. Kerbyson, E. Papaefstathiou, S. C. Perry, J. S. Harper, and D. V. Wilcox, "Pacea toolset for the performance prediction of parallel and distributed systems," *The International Journal of High Performance Computing Applications*, vol. 14, no. 3, pp. 228–251, 2000.

[33] X. Tang, K. Li, G. Liao, K. Fang, and F. Wu, "A stochastic scheduling algorithm for precedence constrained tasks on grid," *Future Generation Computer Systems*, vol. 27, no. 8, pp. 1083–1091, 2011.

[34] J. Sun and J. Liu, "Synchronization protocols in distributed real-time systems," in *Distributed Computing Systems, 1996., Proceedings of the 16th International Conference on*. IEEE, 1996, pp. 38–45.

[35] B. Kao and H. Garcia-Molina, "Deadline assignment in a distributed soft real-time system," *IEEE Transactions on Parallel and Distributed Systems*, vol. 8, no. 12, pp. 1268–1274, 1997.

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

Jiagang Liu (jiagliu@csu.edu.cn) is currently a Ph.D student in the School of Information Science and Engineering, Central South University, China. He is also an associate professor with the School of Computer and Information Science, Hunan Institute of Technology, China. He received the M.Sc. (2007) degrees in computer science from Central South University, China. His research interests include task scheduling, cloud computing, transparent computing.

Ju Ren [S'13, M'16] (renju@csu.edu.cn) received the B.Sc. (2009), M.Sc. (2012), Ph.D. (2016) degrees all in computer science, from Central South University, China. From 2013 to 2015, he was a visiting Ph.D. student in the Department of Electrical and Computer Engineering, University of Waterloo, Canada. Currently, he is a professor with the School of Information Science and Engineering, Central South University, China. His research interests include Internet-of-Things, wireless communication, transparent computing and cloud computing. Dr. Ren serves as an associate editor for IEEE Transactions on Vehicular Technology and Peer-to-Peer Networking and Applications. He has also served as a leading guest editor for IEEE Network, a track co-chair for IEEE VTC'17 Fall and IEEE I-SPAN 2018, a poster co-chair of IEEE MASS'18, a Technical Program Committee member of many international conferences including IEEE INFOCOM'18/19, Globecom'17, WCNC'17 and WCSP'16, and an active reviewer for over 20 international journals. He is a member of IEEE and ACM.

Wei Dai (wdai4@wpi.edu)is a Ph.D. Candidate in the Department of Information Science at University of Arkansas at Little Rock,USA. He has worked for IBM China and IBM China Lab as a senior data scientist engineer for more than seven years. His research interests inlcude machine learning, data quality, IoT and data visualization.

Pude Zhou (pude.zhou@csu.edu.cn) received the B.S. Degree from the School of Electronics Engineering and Computer Science, Lanzhou University of Technology, China, in 2015. He received the M.S. Degree at College of Information Science and Engineering, Central South University. His research interests include big data and cloud computing.

Deyu Zhang [S'14, M'17] (zdy876@csu.edu.cn) received the B.Sc. degree (2005) in communication engineering from PLA Information Engineering University, China, and the M.Sc. degree (2012) from Central South University, China, also in communication engineering. He received his Ph.D. degree in computer science from Central South University, China, in 2016. He is now an assistant professor with the School of Software and a postdoc fellow with the Transparent Computing Lab in the School of Information Science and Engineering, Central South University, China. He was a visiting scholar with the Department of Electrical and Computer Engineering, University of Waterloo, Canada, from 2014 to 2016. He has served as the co-chair of workshop SECIoT'1, and the guest editor for special issue on IEEE Internet of Things Journal. His research interests include stochastic resource allocation transparent computing, edge computing and IoT. He is a member of IEEE and CCF.

Yaoxue Zhang (zyx@csu.edu.cn) received his B.Sc. degree from Northwest Institute of Telecommunication Engineering, China, in 1982, and his Ph.D. degree in computer networking from Tohoku University, Japan, in 1989. Currently, he is a professor in the Department of Computer Science at Central South University, China, and also a professor in the Department of Computer Science and Technology at Tsinghua University, China. His research interests include computer networking, operating systems, ubiquitous/pervasive computing, transparent computing, and big data. He has published over 200 technical papers in international journals and conferences, as well as 9 monographs and textbooks. Currently, he is serving as the Editor-in-Chief of Chinese Journal of Electronics. He is a fellow of the Chinese Academy of Engineering.

Geyong Min (g.min@exeter.ac.uk) is a professor of high performance computing and networking at the Department of Mathematics and Computer Science University of Exeter, United Kingdom. He received the Ph.D. degree in computing science from the University of Glasgow, United Kingdom, in 2003, and the B.Sc. degree from Huazhong University of Science and Technology, China, in 1995. He has published more than 200 research papers in leading international journals including IEEE/ACM Transactions on Networking, IEEE Journal on Selected Areas in Communications, IEEE Transactions on Communications, IEEE Transactions on Wireless Communications, IEEE Transactions on Multimedia, IEEE Transactions on Computers, IEEE Transactions on Parallel and Distributed Systems, and at reputable international conferences, such as SIGCOMM-IMC, ICDCS, IPDPS, GLOBECOM, and ICC. He is an Associated Editor of several international journals, e.g., IEEE Transactions on Computers. He served as the General Chair/Program Chair of a number of international conferences in the area of Information and Communications Technologies. His research interests include future internet, computer networks, wireless communications, high performance computing and cloud computing.

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCC.2019.2906300, IEEE Transactions on Cloud Computing

Noushin Najjari (n.najjari@gmail.com) is a postdoc fellow at the Department of Computer Science, University of Exeter, UK. She received the Ph.D. degree in computer science from University of Exeter, UK, in 2017, and M.Sc. degree in information technology from Sharif University of Technology, Iran, in 2010, and the B.Sc. degree from Alzahra University, Iran, in 2006. Her research interests include performance modelling and analysis of communication networks, next generation networking, wireless network protocols, energy-efficiency, and cross-layer optimization.

2168-7161 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.